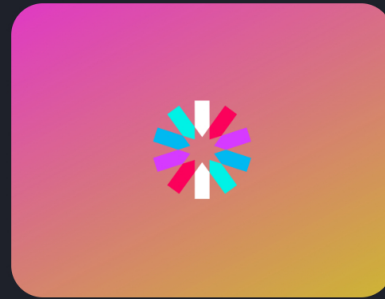
[Developers](#) [Documentation](#) [Product](#) [Solutions](#) [Blog](#)[Pricing](#)[Sign up](#)[Contact sales](#)[← Learn](#)

Get Started with JSON Web Tokens

All you wanted to know about JSON Web Tokens but were afraid to ask.

[Get started →](#)

What is JSON Web Token?

JSON Web Token (JWT) is an open standard ([RFC 7519](#)) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. JWTs can be signed using a secret (with **HMAC** algorithm) or a public/private key pair using **RSA**.

Let's explain some concepts of this definition further.

- **Compact:** Because of its size, it can be sent through an URL, POST parameter, or inside an HTTP header. Additionally, due to its size its transmission is fast.
- **Self-contained:** The payload contains all the required information about the user, to avoid querying the database more than once.

Interested in getting up-to-speed with JWTs as soon as possible?

[DOWNLOAD THE FREE EBOOK](#)

When should you use JSON Web Tokens?

These are some scenarios where JSON Web Tokens are useful:

- **Authentication:** This is the typical scenario for using JWT, once the user is logged in, each subsequent request will include the JWT, allowing the user to access routes, services, and resources that are permitted with that token. Single Sign On is a feature that widely uses JWT nowadays, because of its small overhead and its ability to be easily used among systems of different domains.
- **Information Exchange:** JWTs are a good way of securely transmitting information between parties, because as they can be signed, for example using a public/private key pair, you can be sure that the sender is who they say they are. Additionally, as the signature is calculated using the header and the payload, you can also verify that the content hasn't changed.

Which is the JSON Web Token structure?

JWTs consist of three parts separated by dots (.), which are:

- **Header**
- **Payload**
- **Signature**

Therefore, a JWT typically looks like the following.

```
xxxxx.yyyyy.zzzzz
```

Header

For example:

Then, this JSON is **Base64Url** encoded to form the first part of the JWT.

The second part of the token is the payload, which contains the claims. Claims are statements about an entity (typically, the user) and additional metadata. There are three types of claims: *reserved*, *public*, and *private* claims.

- Notice that the claim names are only three characters long as JWT is meant to be compact.*

- An example of payload could be:

The payload is then **Base64Url** encoded to form the second part of the JWT.

For example if you want to use the HMAC SHA256 algorithm, the signature will be created in the following way.

The following shows a JWT that has the previous header and payload encoded and it is signed with a secret.

ox.1bbCed0d-1f1f-71Nf-TcTpD5cCT6TksVVC-10

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpvaGV9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG91IiwiaXNTb2NpYWwiOiOnRydWV9.4pcPyMD09o1PSyXnrXCjTwXyr4BsezdI1AVTmud2fU4
```

You can browse to jwt.io where you can play with a JWT and put these concepts in practice. jwt.io allows you to decode, verify and generate JWT.

How JSON Web Tokens work?

In authentication, when the user successfully logs in using their credentials, a JSON Web Token will be returned. Since tokens are credentials, great care must be taken to prevent security issues. In general, you should not keep tokens longer than required.

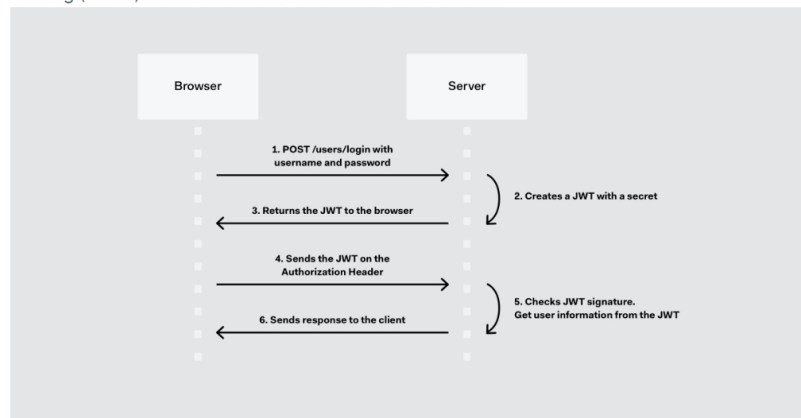
You also [should not store sensitive session data in browser storage due to lack of security](#).

Whenever the user wants to access a protected route, it should send the JWT, typically in the **Authorization** header using the **Bearer** schema. Therefore the content of the header should look like the following.

Authorization: Bearer <token>

This is a stateless authentication mechanism as the user state is never saved in the server memory. The server's protected routes will check for a valid JWT in the Authorization header, and if there is, the user will be allowed. As JWTs are self-contained, all the necessary information is there, reducing the need of going back and forward to the database.

This allows to fully rely on data APIs that are stateless and even make requests to downstream services. It doesn't matter which domains are serving your APIs, as Cross-Origin Resource Sharing (CORS) won't be an issue as it doesn't use cookies.



Why should you use JSON Web Tokens?

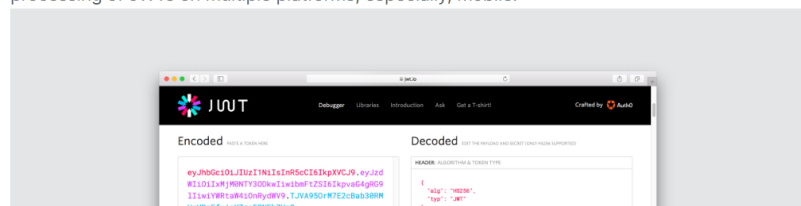
Let's talk about the benefits of **JSON Web Tokens (JWT)** comparing it to **Simple Web Tokens (SWT)** and **Security Assertion Markup Language Tokens (SAML)**.

As JSON is less verbose than XML, when it is encoded its size is also smaller; making JWT more compact than SAML. This makes JWT a good choice to be passed in HTML and HTTP environments.

Security-wise, SWT can only be symmetric signed by a shared secret using the HMAC algorithm. While JWT and SAML tokens can also use a public/private key pair in the form of a X.509 certificate to sign them. However, signing XML with XML Digital Signature without introducing obscure security holes is very difficult compared to the simplicity of signing JSON.

JSON parsers are common in most programming languages, because they map directly to objects, conversely XML doesn't have a natural document-to-object mapping. This makes it easier to work with JWT than SAML assertions.

Regarding usage, JWT is used at an Internet scale. This highlights the ease of client side processing of JWTs on multiple platforms, especially, mobile.



How we use JSON Web Tokens in Auth0?

A screenshot of the Auth0 website footer, which is dark-themed. The footer is organized into four columns. The first column, 'DEVELOPERS', includes links for Developer Hub, Code Samples and Guides, Blog posts, Identity Unlocked - Podcasts, and Zero Index Newsletter. The second column, 'DOCUMENTATION', includes Articles, Quickstarts, APIs, SDK Libraries, Blog, Reports, and Webinars. The third column, 'SUPPORT CENTER', includes Community, Support, Help, FAQs, and Auth0 Marketplace. The fourth column, 'COMPANY', includes Our Customers, Compliance - Ensuring privacy and security, Partners, Careers (with a blue 'We're hiring!' badge), and About us. Below these columns are four more sections: 'GET INVOLVED' (Events, Auth0 Research Program), 'LEARNING' (Learn, Intro to IAM (CIAM)), 'PLATFORM' (Access Management, Extensibility), and 'FEATURES' (Universal Log, Single Sign On). A chatbot bubble from 'Auth0' is overlaid on the bottom right, with a message: 'Hey 👋 It looks like you're interested in Identity and authentication solutions. How can we help?'. The chatbot has a red notification badge with the number '1'.